

Week 4 • Replacing local storage with a real database

Technical Interaction Design • ITU • Autumn 2026

Built 11 September 2026 • commit ea17397

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-04.pdf>

CONTENTS

Backends, Low-Code Backends, and the Parse Platform

- What a backend is
- A low-code backend hands you some of the eight, already built
- Parse is an open-source low-code backend
- You can rent a Parse server, or run your own
- Replacing localStorage with a real database
- Re-implementing the to-do list on top of Parse
- A backend is slow, and your components have to show when the user has to wait for data to be retrieved
- Model your domain before you create a second table
- Further reading

The Three States of Remote Data

- The synchronous case hid the pattern
- A fetch has three states, and your component has to render all three
- `failed` is not the optional one
- The fourth state, and the bug it explains
- What you render in each state is a design decision
- You are going to write this again, and again

Debugging

- `debugger;` is a real statement, and costs nothing when nobody is watching
- React DevTools lets you see state instead of printing it
- Your editor can pause code that is running in the browser
- The stack frame you want is the topmost one in your own file

Backends, Low-Code Backends, and the Parse Platform

Source: [Lectures/Technical/Backend/Backends-Low-Code-Backends-and-the-Parse-Platform.md](#) — something unclear? [Open an issue](#), or send a pull request.

Motivation - we want to be full-stack web developers :) But we don't have much time.

What a backend is

“Backend” only means something relative to a front-end

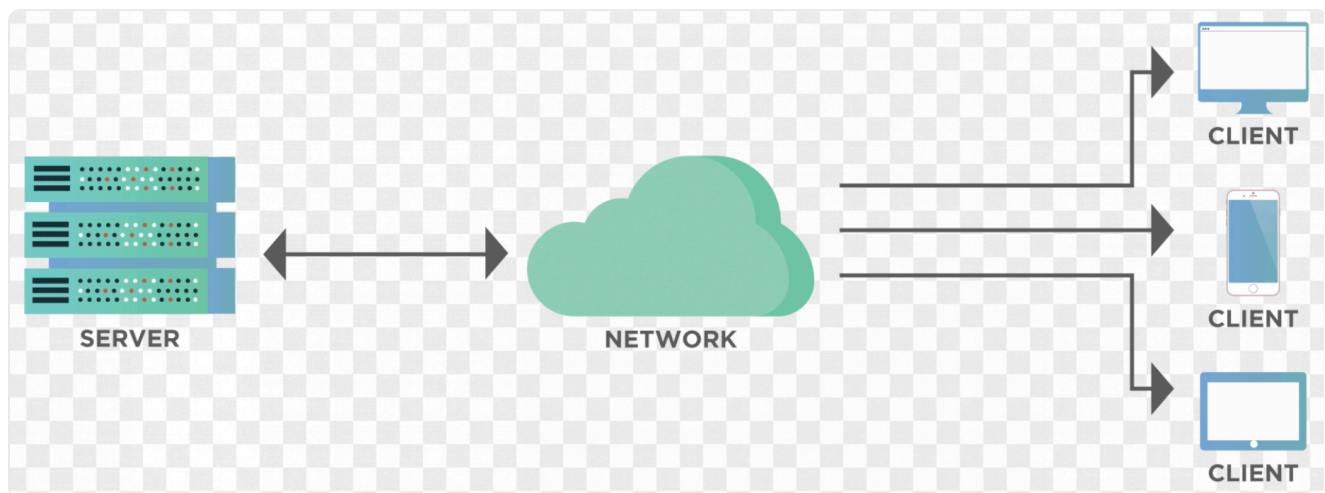
- **Relative term** - defined in opposition to the *front-end*
 - **Front-end** – code that runs in the user's browser and handles presentation and user interaction
 - **Back-end** – handles data processing, storage, and security

Note: Client / front-end are going to be used interchangeably in this lecture

Client-server is the arrangement, and the backend is the server half

The pair has a name, and it is older than the web: a *client-server architecture*.

The **client** runs on the user's machine and asks for things; the **server** runs somewhere else and answers.



The detail that matters is the one the picture shows and the phrase does not: **there are many clients and one server**. Nearly everything in the next three weeks follows from that asymmetry.

- **The server is the only shared thing**, so it is the only place where two users can meet. Sharing a to-do list is possible because there is one copy of it and both browsers are talking to that copy.
- **The server is the only trusted thing**. Every client runs on somebody else's computer, in code they can read and change. Anything you need to be *true* has to be enforced on the server — which is next week's lecture in one sentence.

- **The server is far away.** Between the asking and the answering there is a network, and your interface has to have something to show meanwhile.

The backend owns everything the browser cannot be trusted with

- Authentication (proving that a user is who they say they are)
- Authorization (checking what a user is allowed to do)
- Session management (tracking a user across requests, so they don't log in again on every click)
- Business logic and DB access
- Scheduled jobs (e.g., `cron`, backups, etc.)
- API endpoints / request handling (since the backend receives and responds to requests)
- Data validation (ensuring incoming data is correct/safe)

A traditional backend is eight pieces of infrastructure before a line of your own code

1. **Machine** setup (or create a VM with a cloud provider)
2. **Operating system** installation & configuration
3. Security & **firewall** configuration
4. **Database** management system (DBMS)
5. **Web server** (e.g. nginx, apache2)
6. **Application server** / runtime environment
7. **Logging, monitoring & analytics**
8. **Backup** system

Note that none of the eight is on the previous list. That list was about *responsibilities*; this one is about the *machinery* you would have to assemble before you could discharge a single one of them.

Something unclear in this section? Open an issue.

A low-code backend hands you some of the eight, already built

The infrastructure arrives already assembled, and what you get is an API: pre-built solutions for the needs that every application has, so that the only code you write is the code particular to yours.

The term you will meet in industry is **backend-as-a-service** (BaaS), and it is *not* a synonym for *low-code backend*. **Low-code** says how much you have to write. **As-a-service** says who runs it. Two different questions — and the three names you will hear answer them differently:

- **Parse Platform** is **low-code** — open-source software. You rent a virtual machine in the cloud and install it there, or you run it on your own laptop while you are developing.
- **Firebase** is **as-a-service** — proprietary, hosted by Google; renting it is the only way to have it.
- **Back4App** is **Parse-as-a-service** — somebody else running the low-code backend you could have run yourself.

They are not three of the same kind of thing. Parse is a *program*. Firebase is a *service*. Back4App is a *service that runs the program*.

Which is also why the code you write against Parse does not care: rent it or host it, only the URL changes.

Supabase and **Pocketbase** belong in the same conversation — open source, and deliberately Firebase-shaped alternatives to it. Same idea as Parse, different lineage.

Something unclear in this section? [Open an issue](#).

Parse is an open-source low-code backend

It is open source now, but Parse started as a startup, then got bought by Facebook, and is now open

Startup ⇒ Facebook ⇒ [Open Source](#)

Parse is a Node server

Implemented in JS - runs on Node (remember that JS can run in the browser or on the server via `node`)

Parse also offers you an easy to use Javascript SDK so it's easier for your front-end to communicate with the backend

SDK

- stands for (software development kit)
- represents a library that makes it easy for you to communicate from the frontend to the backend

Everything you do to Parse goes through the JavaScript SDK

- There are also SDKs for other languages than JS (Kotlin for Android, Swift for iOS, etc.)

Parse has an answer for every line of that list

The backend owns...	Parse gives you
Authentication	<code>Parse.User</code> — signup, login, sessions (<i>next week</i>)
Authorization	class-level permissions and ACLs (<i>next week</i>)
Session management	completely free (handled by the SDK, and remembered across page reloads)
Business logic and DB access	the JavaScript SDK — the subject of today
Scheduled jobs	cloud jobs
API endpoints	REST and GraphQL, generated from your classes (although not needed when you use the SDK)
Data validation	cloud code triggers (<i>week 11</i>)

And two more that were never on the list, because the browser never had them to lose: **file storage**, and an interactive **dashboard** for looking at and editing your database by hand.

That is the argument for a low-code backend. Most of the infrastructure is handled, every responsibility on the list has an answer waiting, and what is left for you to write is the part that is actually yours.

Something unclear in this section? [Open an issue.](#)

You can rent a Parse server, or run your own

Back4App hosts Parse for you, and you leave with three keys

Steps to start working with the Back4App Parse deployment

1. Create an account on Back4App
2. Create a backend (app) for your react application in Back4App
3. Somewhere in settings find `APP_ID` and `JAVASCRIPT_KEY` and `PARSE_SERVER_URL` and save them for later

You can host it yourself, and the client code is identical, the only thing that changes is that your client will talk to your own server IP or Domain (e.g. `api.zeeguu.org`) instead of `back4app.com`

- You can also [deploy your own server on DigitalOcean](#)
- I don't recommend it for this course, but if you want you can

Something unclear in this section? [Open an issue.](#)

Replacing localStorage with a real database

Note: The full documentation is in the [Parse.js Javascript Guide](#): use it as reference.

Installing the SDK

```
npm install parse events
npm list parse
```

Check what the second command prints. It should say **parse@8.6.0** — the version we use in the course.

If it prints something older, npm has quietly handed you an old release, because Parse 8 wants Node 20, 22 or 24 and yours is not one of them. npm does not treat that as an error. Run `node --version`, fix Node, delete `node_modules`, and install again.

`parse` is the SDK. `events` provides the browser with `EventEmitter`, a class that Node has built in and the browser does not. Parse's code asks for it; if nobody answers, `Parse.initialize()` fails with `Emitter is not a constructor`. Installing `events` answers.

That failure has a shape you will meet again: `npm run build` **succeeds**, with at most a warning in the output. The app only dies when someone loads it. A green build is not evidence that anything works.

One initialization at the top of the app, and every Parse call knows where to go

```
import Parse from 'parse';

Parse.initialize("YOUR_APP_ID", "YOUR_JAVASCRIPT_KEY");
Parse.serverURL = "https://parseapi.back4app.com/"; // your PARSE_SERVER_URL
```

- By top of the app we mean either `main.jsx` or `App.jsx`
- This is sufficient because `Parse` is a singleton object — every other file that imports `parse` receives the same, already-configured object
- The initialization configures your react application to connect to
 - the server (`Parse.serverURL`)
 - the corresponding app (`YOUR_APP_ID`), because there might be multiple apps on the server

Create and Save An Object to the Database

```
import Parse from 'parse';

const TodoItem = Parse.Object.extend("TodoItem");

const newItem = new TodoItem();
newItem.set("text", "Call the landlord");
newItem.set("done", false);

newItem.save().then(onSave, onError);

function onSave(savedItem) {
  alert("saved a todo with id: " + savedItem.id);
}

function onError(error) {
  alert(error.message);
}
```

Steps:

1. `Parse.Object.extend("TodoItem")` creates a class for the object
2. `save()` - sends the data to the server
3. `save()` returns a *promise*, so we hand it two functions: one for when it worked, one for when it did not
4. The `TodoItem` class is automatically created in the database if it didn't exist - behavior that can be turned off, and which we *will* turn off next week, once you have met your first column created by a typo

The two functions are given names here on purpose. The same code is more often written with both of them inline, and that is how you will see it in the Parse documentation:

```
newItem.save().then(
  (savedItem) => { alert("saved a todo with id: " + savedItem.id); },
  (error) => { alert(error.message); }
);
```

It is the same thing. It is easier to read when the two branches have names, and harder to forget that the second one exists.

A query is a class, some constraints, and a `find()`

```
const TodoItem = Parse.Object.extend("TodoItem");
const query = new Parse.Query(TodoItem);

query.equalTo("done", false);
query.ascending("createdAt");

const results = await query.find();

for (const item of results) {
  console.log(item.id + " - " + item.get("text"));
}
```

Steps:

- Create a class reference
- Create a query object
- Add constraints on the query object
- call `.find()`

`find()` returns a promise as well. From here on we use `await` rather than `.then()`, because the code then reads in the order in which it happens.

References:

- [Query Constraints](#)
- [Queries on Arrays](#)
- [Queries on Strings](#)

Advanced Parse features, for when you need them

- [Atomic counters](#)
- [Atomic arrays](#)

Something unclear in this section? Open an issue.

Re-implementing the to-do list on top of Parse

You already have a working app. It has a list, a form, a checkbox and a delete button, and at the bottom of `ToDoList.jsx` it has this:

```
useEffect(() => {  
  localStorage.setItem("todos", JSON.stringify(todos));  
}, [todos]);
```

One line, and everything is saved. Whenever anything about any to-do changes, the entire array is serialized and written again.

The whole-array write is the first thing that has to go

With `localStorage` that line is free. It is the same machine, the list is a few kilobytes, and nobody notices.

Now put a network in the middle. Ticking one checkbox would upload every to-do you own. Two people editing the same list would overwrite each other wholesale, because each of them is writing the entire list as they last saw it. And there is no operation in the database for *replace everything*; there are operations for one object at a time.

So the single effect that wrote everything becomes **one call per change**:

In the app the user does	In the database
adds a to-do	create one object
ticks a checkbox	save one object
presses Delete	destroy one object
opens the page	find all the objects, once

This is not a Parse rule. It is what having a backend means.

Where the Parse calls go: a service layer

Instead of calling Parse directly from our UI components, we put those calls in their own file — a **service layer**.

Without one, a component both renders the UI *and* talks to the database, which are two responsibilities (see the **Single Responsibility Principle**, in the further reading). Components get longer, and the same four lines of Parse setup get copied into every one of them:

```
// in the component - messy, and about to be duplicated in the next component  
const TodoItem = Parse.Object.extend("TodoItem");  
const item = new TodoItem();  
item.set("text", text);  
item.set("done", false);  
await item.save();
```

With a service layer the component says what it wants:

```
// in the component - clean, simple  
await createTodo(text);
```

Our services folder starts with a single file, `src/services/todoService.js`:

```
import Parse from "parse";

const TodoItem = Parse.Object.extend("TodoItem");

// React is happier with plain JS objects than with Parse objects,
// and this is also where we unify the treatment of `id` with the other fields
function toPlainObject(parseObject) {
  return {
    id: parseObject.id,
    text: parseObject.get("text"),
    done: parseObject.get("done"),
  };
}

export async function fetchTodos() {
  const query = new Parse.Query(TodoItem);
  query.ascending("createdAt"); // oldest first
  const results = await query.find();
  return results.map(toPlainObject);
}

export async function createTodo(text) {
  const item = new TodoItem();
  item.set("text", text);
  item.set("done", false);
  return toPlainObject(await item.save());
}

export async function setTodoDone(id, done) {
  // we have the id, so we don't need to fetch the object before changing it
  const item = TodoItem.createWithoutData(id);
  item.set("done", done);
  return toPlainObject(await item.save());
}

export async function deleteTodo(id) {
  const item = TodoItem.createWithoutData(id);
  await item.destroy();
}
```

`toPlainObject` is the only place in the whole app that knows a field is called `"text"`. Everywhere else, a to-do is `{ id, text, done }` — the same shape it had when it lived in `localStorage`, which

is why the components barely change.

`createWithoutData(id)` builds a reference to an object that already exists on the server, without fetching it. You can then set a field and save, or destroy it, and only that travels over the network.

And the component

The effect that *wrote* everything is replaced by an effect that *reads* everything, once:

```
import { useState, useEffect } from "react";
import { fetchTodos, createTodo, setTodoDone, deleteTodo } from "../services/todoService";

export default function ToDoList({ firstName }) {
  const [todos, setTodos] = useState([]);

  useEffect(() => {
    async function load() {
      setTodos(await fetchTodos());
    }
    load();
  }, []);

  async function handleAdd(newTask) {
    const created = await createTodo(newTask);
    setTodos([...todos, created]);
  }

  async function handleDelete(idToDelete) {
    await deleteTodo(idToDelete);
    setTodos(todos.filter((each) => each.id !== idToDelete));
  }

  async function handleToggle(id) {
    const todo = todos.find((t) => t.id === id);
    await setTodoDone(id, !todo.done);
    setTodos(todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  }

  // ... the JSX is unchanged
}
```

Every handler now does the same two things in the same order: **change the database, then change the state**. The `useEffect` that used to synchronize the two is gone, because there is nothing left for it to synchronize.

The version above is not finished: it assumes the data arrives.

Something unclear in this section? [Open an issue](#).

A backend is slow, and your components have to show when the user has to wait for data to be retrieved

Everything you have fetched so far was instant. `localStorage` is synchronous — the data is already on the machine, so the line after `getItem` has it. That is why this worked:

```
let [todos, setTodos] = useState(loadTodos);
```

A backend is on the other side of a network, and the round trip is somewhere between fifty milliseconds and, on a bad train connection, several seconds. Which means there is now a moment that did not exist before: the component has rendered, and the data has not arrived.

During that moment, `useState([])` gives an empty array, so the list renders — and your app already contains the line that will lie about it:

```
{todos.length === 0 ? <Nothing to do> : ( ... )}
```

The screen says “Nothing to do” when the truth is “I do not know yet.” Those are different things, and the user cannot tell them apart.

Fetches data does not arrive as one state; it arrives as several, and all of them have to be rendered. That is [The Three States of Remote Data](#), and every component you write from here on has them.

Something unclear in this section? [Open an issue](#).

Model your domain before you create a second table

So far there is one class, and no modeling was required: a to-do has a text and a done flag, and that is the whole design. You need to think ahead about the database model as soon as there is more than one kind of thing in your application.

The main questions are

1. What are the types of objects in my domain model?
2. What are the relationships between them?

A second class: to-dos belong to lists

A single flat pile of to-dos stops being useful somewhere around thirty items. What people actually want is *Personal*, *Apartment*, *Bachelor project* — several lists, each with a name. So:

```
erDiagram
    List ||--o{ TodoItem : contains
    List {
        objectId string PK
        name string
    }
    TodoItem {
        objectId string PK
        text string
        done boolean
        list pointer FK
    }
```

Note what we did *not* do: we did not add a `list` string field to `TodoItem`. A string would be enough to group them on screen today, and it would be useless the moment anyone wants to rename a list — or share one, which is next week's whole subject.

And note the three names now in play, each in its own layer: `List` is a class in the database, `ToDoList` is the React component that draws one, and an *array* is a JavaScript value. Different things, so different names.

Obs: the same idea has three names, depending on who is talking

In a relational database	In Parse	In OO lingo
table	class	class
row	object	object
column	field	attribute
foreign key	pointer	a reference
join table	a class with two pointers	—

So yes: a **pointer** is Parse's **foreign key**. It holds which object in which class, and nothing else.

Every relationship you will model is one-to-many or many-to-many

One-to-many relationships are done with pointers

Set a pointer by handing `set()` the whole object, not its id:

```
const List = Parse.Object.extend("List");
const TodoItem = Parse.Object.extend("TodoItem");

const item = new TodoItem();
item.set("text", "Call the landlord");
item.set("done", false);
item.set("list", apartmentList); // ← the object itself, not its id
await item.save();
```

Now we can query all the to-dos in a list:

```
const query = new Parse.Query(TodoItem);
query.equalTo("list", apartmentList);
const results = await query.find();
```

or get the list a to-do belongs to:

```
const list = item.get("list");
```

with one catch that will bite you: what comes back from `get("list")` is a Parse object that knows its `id` but has **not** fetched its fields. Asking it for `get("name")` gives you `undefined`. Either fetch it, or — much better — tell the query to bring the lists along:

```
const query = new Parse.Query(TodoItem);
query.include("list"); // ← fetch the pointed-to objects too
const results = await query.find();
results[0].get("list").get("name"); // now this works
```

One request instead of one-per-to-do. We will have more to say about this in the lecture on efficient communication with the backend.

Many-to-many relationships are done with a join table

Say a to-do can be tagged with several labels, and a label applies to many to-dos. Create a class whose job is to represent one pairing:

```
const TodoLabel = Parse.Object.extend("TodoLabel");

const todoLabel = new TodoLabel();
todoLabel.set("todo", todoItem);
todoLabel.set("label", label);
await todoLabel.save();
```

Two pointers, one per side. That is all a join table is.

Why a join table rather than anything cleverer?

Because the relationship itself will eventually want to carry information, and only a class can hold information:

```
todoLabel.set("addedBy", someUser);
todoLabel.set("addedAt", new Date());
todoLabel.set("order", 1);
```

The moment you need *when* the label was added, or *who* added it, or in *what order*, a join table already has room for it and the alternatives do not.

You will meet `Parse.Relation` in the documentation, which is Parse's built-in way of doing many-to-many. It is less typing and it cannot carry any information about the relationship, so we are not going to use it. Knowing that it exists is enough.

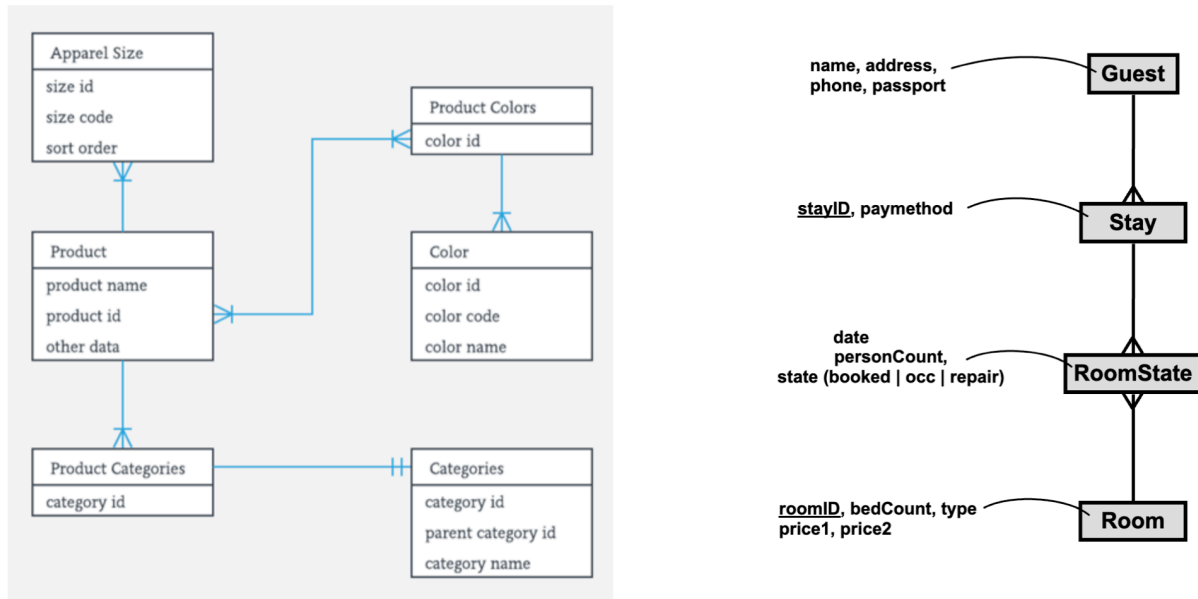
Do not model relationships with arrays

Parse lets you store an array of objects in a field, and it is tempting for small collections. Resist it: an array has no room for information about the relationship, it has to be rewritten in full to add one element — the whole-array problem from the beginning of this lecture, all over again — and it gets slow and awkward as soon as it is not tiny. Pointers for one-to-many, a join table for many-to-many. Those two cover everything you need this semester.

The notation matters less than being able to explain your model

Use whichever notation you prefer. Two that I like are:

1. On the left hand side is the most popular way of showing attributes
 - crow's feet show cardinality
 - attributes are listed in the box
2. On the right hand side is a compressed approach proposed by Søren Lauesen, ex-professor at ITU



No matter which notation you use, the most important aspect is being able to communicate the way all the relevant data for your application domain is saved in the database.

Something unclear in this section? Open an issue.

Project Work

- Design a **domain model** for your application by **creating an ER diagram**. The diagram will be part of your final report. Discuss your diagram with the staff. Make sure to keep it up to date as your project progresses. As you work on your implementation you will realize that you need to constantly refine it. Keep it up to date.
- Create the tables corresponding to your ER diagram in Back4App
- Start connecting your React application to your own Parse backend

Further reading

- SOLID principles
 - **Single Responsibility Principle** - the one behind the service layer
 - Open-closed Principle
 - Liskov Substitution
 - Interface Segregation
 - **Dependency Injection Principle**

Something unclear in this section? Open an issue.

Exam Questions

1. What is the difference between front-end and back-end?
2. List at least 5 responsibilities of a backend.
3. What is Parse Platform and what does it provide?
4. What does CRUD stand for and what do each of the letters represent?
5. Explain what this code does:

```
const TodoItem = Parse.Object.extend("TodoItem");
const newItem = new TodoItem();
newItem.set("text", "Buy milk");
newItem.set("done", false);
await newItem.save();
```

6. This code was fine when the to-dos were in `localStorage`. Why is it no longer acceptable once they live on a server, and what replaces it?

```
useEffect(() => {
  localStorage.setItem("todos", JSON.stringify(todos));
}, [todos]);
```

7. What are two benefits of placing this code in a separate service file rather than directly in a React component?

```
export async function fetchTodos() {
  const query = new Parse.Query(TodoItem);
  query.ascending("createdAt");
  const results = await query.find();
  return results.map(toPlainObject);
}
```

8. What is wrong with this query pattern, and which single line fixes it?

```
const query = new Parse.Query("TodoItem");
const todos = await query.find();

for (let todo of todos) {
  const list = todo.get("list");
  await list.fetch();
  console.log(list.get("name"));
}
```

9. How would you query all TodoItems where done is false, ordered by creation date?

10. Why is a join table preferred over an array field for a many-to-many relationship?

References

The documentation on ParsePlatform.org

- [Getting Started Guide](#) - extensive reference for everything ParseJS
- [Relationships](#) - this is very good and must be read attentively – it will really help with modeling

The Three States of Remote Data

Source: [Lectures/Technical/React/The-Three-States-of-Remote-Data.md](#) — something unclear? [Open an issue](#), or send a pull request.

Up to now, every component that had data simply had it. This note is about what happens to a component when its data lives somewhere else, and the answer is not “it waits” — components cannot wait. It renders anyway, and you have to decide what it renders.

The synchronous case hid the pattern

Here is how the to-do list loaded its data when the data was in `localStorage` :

```
let [todos, setTodos] = useState(loadTodos);
```

A `useState` initializer has to **return the value**. `loadTodos` could, because `localStorage.getItem` is synchronous: the data is already on the machine, so the line after the call has it.

Reading the data and having the data happen at the same instant. There is no in-between moment, so there is nothing to render during it, so the component has exactly one state: the data.

This is a special case, and it is the only one you have met so far. It stops being available the moment the data is on a server. `fetchTodos()` cannot be a `useState` initializer, because it does not return to-dos — it returns a promise, and React would put the promise in your state.

Something unclear in this section? [Open an issue](#).

A fetch has three states, and your component has to render all three

The initial state can now only be “nothing yet”, and the data has to arrive later, which is why it arrives through an effect. And the arrival itself has to become state, because the component needs to render differently before and after it:

```
function ToDoList() {
  const [todos, setTodos] = useState([]);
  const [status, setStatus] = useState("loading");

  useEffect(() => {
    async function load() {
      try {
        setTodos(await fetchTodos());
        setStatus("ready");
      } catch (error) {
        console.error(error);
        setStatus("failed");
      }
    }
    load();
  }, []);

  if (status === "loading") return <p>Loading your to-dos...</p>;
  if (status === "failed") return <p>Could not reach the server. Try again.</p>;

  return (
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
}
```

Loading, failed, ready. Three screens, from one fetch.

The rendering itself is nothing new — it is the early-return form of [conditional rendering](#), which is why that note showed you three shapes and said the `if` was the one for whole-screen decisions. This is the screen it meant.

Something unclear in this section? [Open an issue](#).

failed is not the optional one

It is the one people leave out, because during development the server is on your own machine and it never fails.

Without it, a dead network looks exactly like an empty list. The user retries nothing, because as far as they can see nothing went wrong. A `catch` that only writes to the console is a bug with a comforting shape: you see the error while developing, and your users never do.

Something unclear in this section? [Open an issue.](#)

The fourth state, and the bug it explains

A `ready` list can still have nothing in it, and “*you have no to-dos*” is a real screen that a real app needs. It is often the nicest screen in the app — the one that tells a new user what to do first.

So the states are really:

state	what it means	what the user should see
loading	I have not heard back yet	a skeleton, or a quiet message
failed	I asked, and it went wrong	what went wrong, and a way to retry
empty	I heard back: there is nothing	an invitation to add the first one
ready	I heard back: here it is	the data

And now the bug in the to-do list has a name. The app already had an empty state:

```
{todos.length === 0 ? <Nothing to do> : ( ... )}
```

When the data moved to a server, no loading state was added — so **the empty state impersonated the loading state**. `useState([])` gives an empty array, `length === 0` is true, and the screen cheerfully says *Nothing to do* when the truth is *I do not know yet*. Those are different things, and the user cannot tell them apart.

This is the general shape of the mistake: when a state is missing, some other state takes its place, and it is usually the one that looks most like success.

Something unclear in this section? [Open an issue.](#)

What you render in each state is a design decision

`<p>Loading...</p>` is honest and takes a minute. A spinner is a rotating element in CSS, and there is nothing React-specific about it. Better than either, when you know the shape of what is coming, are **skeleton rows** — grey blocks the size of the real to-dos — because the layout does not jump when the data lands.

A spinner has a failure mode of its own: if the data usually arrives in 200ms, it appears and disappears before the eye resolves it, and the flicker reads as a glitch rather than as progress. Showing nothing for the first few hundred milliseconds often looks *faster* than showing a spinner immediately.

And for `failed`, the word “Retry” on a button is worth more than any amount of apology in the message. An error the user can do something about is a smaller error.

Something unclear in this section? [Open an issue](#).

You are going to write this again, and again

Every screen that fetches needs these states, which means the same pair of `useState` calls in every component that talks to the backend. Duplication that shows up in every component of a certain kind is usually telling you something, and later in the course we come back and do something about this one.

Something unclear in this section? [Open an issue](#).

Exam Questions

1. Why could `useState(loadTodos)` load the to-dos from `localStorage`, but cannot load them from a backend?
2. Name the three states a fetching component has, and say what the user sees in each.
3. This component fetches in a `useEffect` and starts with `useState([])`. What does the user see before the data arrives, and why is it wrong?
4. What is the difference between the `empty` state and the `loading` state, and what goes wrong when a component has only one of them?
5. Why is it not enough to `console.error` in the `catch`?

Debugging

Source: [Lectures/Technical/React/Debugging.md](#) — something unclear? [Open an issue](#), or send a pull request.

Three tools, in the order you will reach for them: a `debugger;` statement, the React DevTools component tree, and a proper breakpoint in your editor. Then a word on reading the error before you touch anything.

`debugger;` is a real statement, and costs nothing when nobody is watching

`debugger;` is a real JavaScript statement, not a library call. Put it anywhere in your code: it does nothing while nobody is watching, and pauses execution the moment DevTools is open.

That makes it the cheapest breakpoint there is — and unlike a `console.log`, you cannot forget where you left it, because it stops the program and shows you.

Something unclear in this section? [Open an issue](#).

React DevTools lets you see state instead of printing it

[React Developer Tools](#) is a browser extension — install it once, for Chrome, Firefox or Edge, and it adds two panels that plain DevTools does not have:

- **Components** — the live component tree, with every component's props and state visible and editable. This is the only way to see state rather than print it, and it is what turns "the list did not update" into "the state changed but the [key](#) did not", which are different bugs.
- **Profiler** — which components re-rendered and why. Ignore this until something is slow; it answers a question you do not have yet.

Something unclear in this section? [Open an issue](#).

Your editor can pause code that is running in the browser

In VS Code, this takes a one-off configuration.

1. Create the `.vscode` folder in your project
2. Add the following configuration in the `launch.json` file

```
{
  "configurations": [
    {
      "type": "chrome",
      "request": "launch",
      "name": "Debug localhost in Chrome ",
      "url": "http://localhost:5173/",
      "webRoot": "${workspaceFolder}/src"
    }
  ]
}
```

3. Set your breakpoints

4. Select from the menu: `Run → Start Debugging`

Something unclear in this section? [Open an issue.](#)

The stack frame you want is the topmost one in your own file

Most React errors name the component and the hook. Read the first three lines of the stack before changing anything: the frame you want is usually the topmost one that belongs to *your* file rather than to `react-dom`.

Something unclear in this section? [Open an issue.](#)

Exam Questions

1. What does `debugger`; do when nobody has DevTools open, and why does that make it safer to leave in than a `console.log`?

2. “The list did not update.” Name two different bugs that look exactly like that on screen, and say how the Components panel tells them apart.

3. An error’s stack is fifteen frames deep and most of them are inside `react-dom`. Which frame do you read first, and why that one?